



SAP HANA Database Backint Specification

1.50.2

Target Audience

- Development Partners

©2025 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

The information contained herein may be changed without prior notice. Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies. Please see <https://www.sap.com/about/legal/trademark.html> for additional trademark information and notices.

Contents

1 Document History	4
2 Introduction	6
2.1 Overview	6
2.2 Platforms	6
3 Interface Definition	7
3.1 Overview	7
3.2 Command Line Options	7
3.3 Return Codes	9
3.4 Input and Output Files	9
3.5 Names	9
3.6 Multiple Hosts	10
3.7 Description of Functions	10
3.7.1 Backup	10
3.7.2 Restore	11
3.7.3 Inquire	12
3.7.4 Delete	12
3.8 Parameter File	13
3.9 Backup Levels	13
3.10 Number of Database Objects	13
4 Installation	14
5 Definitions	15
6 Examples	16
6.1 Log Backup	16
6.2 Data Backup	17
6.3 Log Restore	18
6.4 Inquiring Objects from Backint	19

1 Document History

1.00

- Initial version

1.01

- add session options: -l, -c
- add chapter Examples
- new definitions of parameter length
- removed inquire command: `#EBID <external_backup_id>`
- add parameter for all functions: `#SOFTWAREID`
- add optional parameter `#TOOLOPTION`
- add option -v and -V for version info

1.02

- Fix some typos, layout etc.

1.03

- changed formatting

1.04

- clarify return code handling
- re-add Definitions section
- add Installation section
- changed pipe location in examples
- add multi pipe restore log example

1.05

- clarify multi-host behavior

1.06

- fixed path description for hdbbackint and hdbconfig

1.07

- migrated document source from Doxygen to LaTeX
- updated documentation for options -p and -l
- updated documentation for options -c
- updated legal notice disclaimer
- fixed a few typos

1.50

- extensions to protocol for major backint version 1.5
- database name is also passed to backint with -u
- maximum size of the object is provided with "backup" function
- new backup types with -l
- "inquire" function returns creation timestamp of the object

1.50.1

- update formatting
- clarified handling for reading from pipes

1.50.2

- add note also for writing to pipes

2 Introduction

2.1 Overview

This document describes the interface *Backint for SAP HANA*. This interface enables third-party backup tool vendors to use their backup tools in combination with SAP HANA.

Backint for SAP HANA (from here on referred to as Backint) is an executable which is supplied by third-party backup tool vendors. When a database backup is started, this executable is called by the SAP HANA database. Backint is responsible for executing all activities needed to transfer the backup data to a secure storage location. The data to be backed up is read by the backup tools from pipes. These pipes are supplied by the database.

The biggest difference to conventional database systems is that the SAP HANA database is a distributed database. This has implications for the backup concept. Every active host in the database may have one or more volumes to be backed up. Several Backint processes will be started for a data backup which may have to handle one or more backup objects. A backup object can be a file in the file system or a named pipe. When a backup operation is performed, a unique *database backup id* is passed to Backint to uniquely identify backup objects that belong together. In addition, the number of database objects associated to that database backup id can also be specified. This way, the Backint is able to determine when an operation with multiple Backint calls has completed successfully.

Additionally, a log backup may be started in parallel independently for every volume. Also, every log backup is assigned a unique database backup id.

All these concurrently served pipes have to be handled independently from each other. This means that no operation on an arbitrary pipe may be blocked by any not yet executed operation on another pipe.

As the database also supports failover between hosts, the set of hosts may vary between backups and recovery operations. Thus a recovery may be started on a different host than the one where the backup was initiated. This makes it necessary to have access through Backint from every host to every backup server.

2.2 Platforms

Backint for SAP HANA is supported on all released platforms. More information: Platform Availability Matrix (PAM) <https://service.sap.com/pam>

Backint 1.5 is supported on HANA SPS05 and upper releases.

3 Interface Definition

3.1 Overview

To execute backup and recovery Backint needs to be provided with additional information. This information is passed by command line options and different types of files. The files contain more detailed information – either as input for Backint or as a result for the caller of Backint.

3.2 Command Line Options

The following command line options have to be supported:

```
backint
  -u <user_id>
  [ -f <function> ]
  [ -p <par_file> ]
  [ -i <in_file> ]
  [ -o <out_file> ]
  [ -s <database_backup_id> ]
  [ -c <number_of_objects> ]
  [ -l <backup_level> ]
  [ -v ]
  [ -V ]
```

The following section describes these options in detail:

Parameter	Description	Values
-u <user_id>	The <i>user id</i> defines a common name space for file names in a multi-host SAP HANA database. The user id is bound to a database instance and used by all the servers in this instance. It consists of the name of the database from which the backint operation is invoked and system id of the HANA instance	DB-NAME@SID
-f <function>	The requested function.	backup, restore, inquire, delete
-p <par_file>	Name of the vendor supplied parameter file (see section Parameter File)	-
-i <in_file>	The name of the input file containing the parameters for the corresponding function (-f). If this option is not set, the input is read from stdin.	-
-o <out_file>	The name of the output file which is used by Backint to write return values and messages. If this option is not set, the output is written to stdout.	-
-s <database_backup_id>	The database backup id which can be used to determine which calls to Backint belong to a specific multi-file backup. This option is only usable if the function is backup.	-
-c <number_of_objects>	The total number of database objects associated to the database backup id specified with -s. This option is only usable if the function is backup and in combination with option -s.	-
-l <backup_level>	The backup level indicates the type of backup. This option is only usable if the function is backup.	COMPLETE, DIFFERENTIAL, INCREMENTAL, METADATA, INITIALVOLUME, CATALOG, LOG
-v	This option requests the version to be printed to stdout. One line describing Backint API version and Backint implementation version. It should be the same string as #SOFTWAREID in the output file. For example: "backint 1.50" "ACME Corp backint 0815.2020.02".	-
-V	Detailed version information, same as text as for option -v plus additional lines with detailed version and/or licensing information.	-

SAP HANA database does not rely on any action related to the parameters passed with options -s, -c or -l. These are for informational purposes only.

The options -v and -V will never be called by SAP HANA database, they are for diagnosis only. Implementation of -V is optional.

3.3 Return Codes

The return codes of Backint are defined as follows:

Value	Symbol	Description
0	OK	Backint ran without problems, the results of the function are listed in the output file. If a function returns <code>#NOTFOUND</code> , <code>#NOTDELETED</code> or <code>#ERROR</code> for one or more backup objects it should return with 0, these values are <i>expected</i> .
1	ERROR	If Backint cannot start properly (syntax error in command line, communication problems and the like) or cannot write a regular output file, it should terminate with a return code different from 0. The content of the output file will not be parsed and the whole function will be interpreted as failed.

The content of the output file, the Backint output on stdout and stderr and the return value are stored by the SAP HANA database to support error analysis.

3.4 Input and Output Files

The input file contains information necessary to perform the requested function `-f`. It usually contains a list of objects.

The output file contains result information for the database. Each line must start with a keyword defined in the following sections. Lines not starting with a keyword are considered as comments and may contain any plain text e.g. to indicate progress or to support error analysis. The character encoding of both files is UTF-8.

There are some formatting requirements for the parameters following keywords:

- Parameters may be quoted with double quotation marks `"`.
- Parameters containing a space must be quoted with double quotation marks.
- If a parameter contains a double quote, the double quote must be escaped with a backslash.
- A backslash escapes double quotes only. A backslash in a parameter value must not be escaped.

Below are examples of valid input or output lines:

```
#PIPE /var/tmp/backup_Monday_cw47_2012 8192
#PIPE "/var/tmp/backup Monday cw47 2012" 8192
#EBID "Hb12\"NAB" "/var/tmp/myTool.#YSOwa"
```

Input files with multiple lines handling objects via pipe like `#PIPE` in function `BACKUP` or `#EBID` and `#NULL` in function `RESTORE` must be processed in sequential order starting from top to bottom. One Backint may handle multiple pipes in parallel but should not skip entries for later processing or pause processing of entries depending on events in subsequent entries.

Input and output files are provided to Backint by the SAP HANA database.

3.5 Names

The database creates a unified namespace for files or pipes on all hosts in a SAP HANA instance identified by the *user id*. If an object with the same *file name* and user id of an existing object is requested for backup

on a different host, it should be treated as a new version of the existing object. Restore of an object can be requested on any host of a SAP HANA instance regardless of the host of origin.

3.6 Multiple Hosts

Backup and recovery of a SAP HANA database is based on backups of individual volumes. Each volume is identified by a permanent unique volume identifier and is associated with a database service. From a backup perspective, services are operating system processes operating on data and log volumes, thus a data backup contains the payload of a data volume that is associated with the service. In a multi-host SAP HANA database volumes are not bound to a specific host. They are globally visible and the associated services might run on different hosts with every restart, failover or recovery.

This concept can lead to a situation where a data backup created on one host may be restored on another. Furthermore a log recovery of one service may contain log backup objects that were created before on more than one host. Therefore, the API functions INQUIRE, DELETE and RESTORE need particular attention in case of a multi-host system.

The function INQUIRE issued on a host must consider all backup objects of all hosts of the database that fit to the request. The function DELETE may delete multiple backup objects of different services created on different hosts with one single delete request. The function RESTORE may request multiple backup objects of one service created on different hosts with one single call. A RESTORE request containing #NULL together with the backup object identifier must consider the latest version of the specified backup object regardless on which host the specified backup object was created before.

3.7 Description of Functions

The function used on the command line (option -f) is applied to a list of objects specified in the input file. Each of these objects is described by a line complying with the syntax rules indicated below. Additionally each input file contains at least the current API and software version and, optionally, a parameter for options passed to Backint on a per-call basis. The option string is known by the database, for example, as a parameter to the backup SQL command. The option string is passed to Backint transparently.

```
#SOFTWAREID <backint_version> <software_version>
#TOOLOPTION <option_string>
```

Each output file must contain at least the current API and software version of the Backint implementation:

```
#SOFTWAREID <backint_version> <software_version>
```

There are 2 major backint API versions ("1.0" and "1.5"). The digit after the major version implies the minor version. The full version consists of 3 digits e.g. "1.04", "1.50"

The backup tool must communicate with HANA by using the the same major backint API version that is used by HANA. That's why the major API version reported in the output file must be same with the version given in the input file.

3.7.1 Backup

For backup, the input file contains any number of the two types of entries: #PIPE and #FILE. They contain one or more descriptions of pipes or files to be used by Backint. Each line starts with an indicator

that defines the type of the data source, followed by the name of the pipe or file in the file system and finally the maximum size of data that will be provided via the pipe or file:

```
#PIPE <file_name> <max_size>
#FILE <file_name> <max_size>
```

The provided maximum size information is optional. It will be provided for the native HANA volumes for data, log and catalog backups but it will not be provided for HANA Dynamic Tiering volumes.

The external backup tool has to retain the information about the type. This information is used in a subsequent restore. Nonexistent or unreadable files or pipes should be treated as an error.

As previously mentioned, data may be written to pipes in any order. The backup tool must not stop reading from an individual pipe on its own initiative, for example, due to an arbitrary timeout. The database will continue to send data on at least one pipe until the backup is complete. A timeout condition for the backup tool is only applicable if no data is sent on any pipe at all.

Analogously, in recovery scenarios the backup tool must not time out writing to individual pipes. The database will continue to receive data on at least one pipe until the restore process is complete.

For each entry in the input file, a corresponding entry is returned in the output file. This entry may indicate success or an error:

```
#SAVED <external_backup_id> <file_name> <size>
#ERROR <file_name>
```

3.7.2 Restore

When a restore is performed, Backint is supplied with more information than for a backup because, for a restore, a specific backup from the backup server has to be selected. Thus not only the 'file name' of the pipe or file is given, but also the 'external backup id' which denotes the selected backup. If `#NULL` is specified instead of the external backup id, the newest available backup for this file name has to be selected. The type of backup (file or pipe) is stored in the backup server and is used to select the correct file object during restore. During restore, files are created by Backint. Pipes are created in advance by the database.

If you want to restore the content to another file name, you can add the absolute name of a target to the combination of external backup id and the name of the file or pipe.

A specific backup is determined by its external backup id together with its name in the file system. If `#NULL` is specified, the latest version of the object, identified by the file name, is retrieved. Thus every line in the input file has the following format:

```
#NULL <file_name> [<dest_name>]
#EBID <external_backup_id> <file_name> [<dest_name>]
```

For every object in the input file, the result is returned in a corresponding line in the output file. A successful restore is indicated by the keyword `#RESTORED`; an error is indicated by the keyword `#ERROR`. If the requested backup object could not be found on the backup server, a message starting with `#NOTFOUND` is returned.

```
#RESTORED <external_backup_id> <file_name>
#ERROR <file_name>
#NOTFOUND <file_name>
```

3.7.3 Inquire

The function `inquire` is used to obtain information from the backup server. The input file contains information about the objects for which information is requested. The input file contains multiple lines in one of the following three formats:

```
#NULL
#NULL <file_name>
#EBID <external_backup_id> <file_name>
```

For every object in the input file, the result is returned in at least one corresponding line in the output file. If an inquire matches multiple objects, a line is written for every object to the output file, sorted by the date the backup was created (latest backup first). The inquiry `#NULL` lists all the available database objects for the specified user id.

- A matching database object is indicated by the keyword `#BACKUP`. The entry must contain the EBID of the object, the file name and the creation timestamp of the object. The timestamp must be reported in the ISO 8601 format, in UTC timezone with milliseconds precision and in quotations(i.e. "2020-02-06T08:02:44.100Z").
- If no matching database objects were found on the backup server, a message starting with `#NOTFOUND` is returned.
- Errors are indicated by the keyword `#ERROR`.

```
#BACKUP <external_backup_id> <file_name> "<creation_timestamp>"
#ERROR [<external_backup_id>] [<file_name>]
#NOTFOUND [<external_backup_id>] [<file_name>]
```

3.7.4 Delete

In order to support housekeeping and the cleanup of incomplete or erroneous backups the function `delete` has to be supported. The input file contains the information about the backup to be deleted in the following form:

```
#EBID <external_backup_id> <file_name>
```

For every object in the input file the result is returned in a corresponding line in the output file.

- A successful deletion is indicated by the keyword `#DELETED`.
- An error is indicated by the keyword `#ERROR`.
- If the requested backup object could not be found on the backup server, a message starting with `#NOTFOUND` is returned.
- If the backup server denied a deletion, this is indicated by the keyword `#NOTDELETED`.

```
#DELETED <external_backup_id> <file_name>
#NOTDELETED <external_backup_id> <file_name>
#ERROR <external_backup_id> <file_name>
#NOTFOUND <external_backup_id> <file_name>
```

If SAP HANA database has written multiple database objects with the same database backup id, it will delete all of these objects with one Backint call.

3.8 Parameter File

The parameter file is any file, which contains third-party tool vendor-specific information that is needed for backup or recovery. As this is vendor-specific, it cannot be supplied by the database. For example, this file may contain information about the backup tool client installation or connection information for the backup server. The file location is known by the database and passed to Backint. Parameter files can be assigned individually for data backups, log backups, and catalog backups. It is also possible to assign individual parameter files to each host and to each server.

3.9 Backup Levels

The backup level of an operation depends on the type of backup in question. The available backup levels are defined as follows:

Backup Type	Backup Level
Complete Data Backup	COMPLETE
Differential Data Backup	DIFFERENTIAL
Incremental Data Backup	INCREMENTAL
Initial Volume Backup	INITIALVOLUME
Catalog Backup	CATALOG
Metadata Backup	METADATA
Log Data Backup	LOG

3.10 Number of Database Objects

For backup operations, the option `-c` can be used to specify the total number of database objects associated to the database backup id given by `-s`. This section describes aspects to consider when using this option.

Catalog Backups Each data backup and log backup may be followed by a catalog backup. Catalog backups are always conducted in a distinct backup operation. As a result, each catalog backup features a unique database backup id. Since catalog backups consist of a single database object, the number of database objects specified using `-c` is equal to 1.

Multistream Backups When making use of multistream backups, multiple pipes are used for transferring single database objects. Multistreaming has no influence on the number of database objects specified using `-c`.

4 Installation

The third-party backup tool supporting the Backint for SAP HANA interface should be installed on the same shared file system as the SAP HANA database. `<sharedDrive>/opt/backint/` should be the base directory of the third-party backup tool installation. Multiple software versions or SID-specific installations may be installed using a tool specific directory structure inside of `<sharedDrive>/opt/backint/`.

The SAP HANA database expects the Backint executable in `/usr/sap/<SID>/SYS/global/hdb/opt/hdbbackint`.

During the installation of the third-party backup tool a symbolic link pointing from `/usr/sap/<SID>/SYS/global/hdb/opt/hdbbackint` to the location of the executable must be created. Parameter files should be symbolic links pointing from `/usr/sap/<SID>/SYS/global/hdb/opt/hdbconfig/` to the actual parameter files in the directory.

5 Definitions

Type	Description
backint_version	Version of the Backint API: "backint 1.50".
software_version	A name identifying the software. The maximum length of the name is 512 bytes.
option_string	A string passed transparently to Backint to specify additional parameters to Backint. The maximum length of the option string is 512 bytes.
database_backup_id	A database backup id is a unique identifier which is the same for all database objects in the same backup set. This id is created by the database. The maximum length of the id is 32 bytes.
external_backup_id	An external backup id is created by the external backup tool. The maximum length of the id is 32 bytes. The combination of <code>external_backup_id</code> and <code>file_name</code> is required to identify a unique database object.
in_file	A filename in the file system. The maximum length of the name is 512 bytes.
out_file	A filename in the file system. The maximum length of the name is 512 bytes.
par_file	A filename in the file system. The maximum length of the name is 512 bytes.
file_name	The absolute name of a pipe or file in the file system. The maximum length of the name is 512 bytes.
dest_name	The absolute name of the destination in the file system. The maximum length of the name is 512 bytes.
creation_timestamp	The creation timestamp of the object. It must be in ISO 8601 format, in UTC timezone with milliseconds precision and in quotations(i.e. "2020-02-06T08:02:44.100Z")
max_size	Maximum size of the transferred data that will be provided by HANA in bytes. The maximum length is 20 digits (this allows the use of 64 bit unsigned integers).
size	Size of the transferred data in bytes. The maximum length is 20 digits (this allows the use of 64 bit unsigned integers).
user_id	The user id consists of database name and HANA system id and is in the format of <code>dbname@SID</code>

6 Examples

6.1 Log Backup

Log backups are written independently by all servers that have a persistent volume. Each log backup consists of a single database object. A log backup may be followed by a catalog backup. As shown in Section 3.10, catalog backups are performed in a separate backup operation.

For a log backup, the call to Backint may look as follows:

```
backint -u BR1@BR1 -f backup -p aBackint.par -i /var/tmp/  
backint_i_aw9H3P -o /var/tmp/backint_o_aw9H3P -s 42342311 -c 1 -l  
LOG
```

Input file /var/tmp/backint_i_aw9H3P:

```
#SOFTWAREID "backint 1.50" "hdb indexserver 2.00.50"  
#PIPE "/usr/sap/BR1/SYS/global/hdb/backint/log_backup_6_0_243987_246921"  
87721121
```

Result in output file /var/tmp/backint_o_aw9H3P:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"  
#SAVED "DF222xra" "/usr/sap/BR1/SYS/global/hdb/backint/  
log_backup_6_0_243987_246921" 87721121
```

If a catalog object is written afterwards then the corresponding call to Backint may look as follows:

```
backint -u BR1@BR1 -f backup -p aBackint.par -i /var/tmp/  
backint_i_aH32M1 -o /var/tmp/backint_o_aH32M1 -s 55837152 -c 1 -l  
CATALOG
```

Input file /var/tmp/backint_i_aH32M1:

```
#SOFTWAREID "backint 1.50" "hdb nameserver 2.00.50"  
#PIPE "/usr/sap/BR1/SYS/global/hdb/backint/log_backup_0_0_0_0 32768
```

Result in output file /var/tmp/backint_o_aH32M1:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"  
#SAVED "Gh3555a.foo.bar" "/usr/sap/BR1/SYS/global/hdb/backint/  
log_backup_0_0_0_0" 32768
```


6.2 Data Backup

A data backup is a set of multiple database objects. It consists of one database object for each server that has a persistent volume and one metadata object per database. The metadata object is written for each database in advance. Once the metadata object has been successfully written, each server will concurrently invoke the Backint in order to write its individual database object. During this operation all calls to Backint have the same values for `-u`, `-f`, `-p`, `-s`, `-c`, and `-l`. Afterwards, a catalog object may be written. As shown in Section 3.10, catalog backups are performed in a separate backup operation.

First, the metadata object is written by the master service of the target database:

```
backint -u BR1@BR1 -f backup -p aBackint.par -i /var/tmp/backint_i_c3po
-o /var/tmp/backint_o_r2d2 -s 3938779026 -c 3 -l METADATA
```

Input file `/var/tmp/backint_i_c3po`:

```
#SOFTWAREID "backint 1.50" "hdb nameserver 2.00.50"
#TOOLOPTION "Key1:value, Key2:othervalue"
#PIPE "/usr/sap/BR1/SYS/global/hdb/backint/FULL_BACKUP_databackup_0_1"
32768
```

Result in output file `/var/tmp/backint_o_r2d2`:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"
#SAVED "DZM22aiD" "/usr/sap/BR1/SYS/global/hdb/backint/
FULL_BACKUP_databackup_0_1" 32768
```

When the metadata object has been written successfully, each server invokes Backint in order to write its individual database object. This happens concurrently, hence the order of Backint invocations and subsequent outputs is non-deterministic.

For one service, the Backint invocation may look as follows:

```
backint -u BR1@BR1 -f backup -p aBackint.par -i /var/tmp/backint_i_kk321
-o /var/tmp/backint_o_kk322 -s 3938779026 -c 3 -l COMPLETE
```

Input file `/var/tmp/backint_i_kk321`:

```
#SOFTWAREID "backint 1.50" "hdb nameserver 2.00.50"
#TOOLOPTION "Key1:value, Key2:othervalue"
#PIPE "/usr/sap/BR1/SYS/global/hdb/backint/FULL_BACKUP_databackup_1_1"
427744310358
```

Result in output file `/var/tmp/backint_o_kk322`:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"
#SAVED "DZM22aiE" "/usr/sap/BR1/SYS/global/hdb/backint/
FULL_BACKUP_databackup_1_1" 427744310358
```

For another service, the Backint invocation may look as follows:

```
backint -u BR1@BR1 -f backup -p aBackint.par -i /var/tmp/backint_i_kk323  
-o /var/tmp/backint_o_kk324 -s 3938779026 -c 3 -l COMPLETE
```

Input file /var/tmp/backint_i_kk323:

```
#SOFTWAREID "backint 1.50" "hdb indexserver 2.00.50"  
#TOOLOPTION "Key1:value, Key2:othervalue"  
#PIPE "/usr/sap/BR1/SYS/global/hdb/backint/FULL_BACKUP_databackup_2_1"  
618122183
```

Result in output file /var/tmp/backint_o_kk324:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"  
#SAVED "HOIAaiE" "/usr/sap/BR1/SYS/global/hdb/backint/  
FULL_BACKUP_databackup_2_1" 618122183
```

If a catalog object is written afterwards then the corresponding call to Backint may look as follows:

```
backint -u BR1@BR1 -f backup -p aBackint.par -i /var/tmp/backint_i_hvsdl  
-o /var/tmp/backint_o_sdlhv -s 4059567531 -c 1 -l CATALOG
```

Input file /var/tmp/backint_i_c3po:

```
#SOFTWAREID "backint 1.50" "hdb nameserver 2.00.50"  
#TOOLOPTION "Key1:value, Key2:othervalue"  
#PIPE "/usr/sap/BR1/SYS/global/hdb/backint/log_backup_0_0_0_0" 32768
```

Result in output file /var/tmp/backint_o_r2d2:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"  
#SAVED "1001231" "/usr/sap/BR1/SYS/global/hdb/backint/log_backup_0_0_0_0"  
" 32768
```

6.3 Log Restore

During a recovery, multiple objects are read by the database. The recovery is started by the name server which restores the latest version of a special database object.

```
backint -u BR1@BR1 -f restore -p aBackint.par -i /var/tmp/  
backint_i_gkn321 -o /var/tmp/backint_o_gkn322
```

Input file /var/tmp/backint_i_gkn321:

```
#SOFTWAREID "backint 1.50" "hdb nameserver 2.00.50"  
#NULL "/usr/sap/BR1/SYS/global/hdb/backint/log_backup_0_0_0_0"
```

Result in output file /var/tmp/backint_o_gkn322:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"
#RESTORED "1001231" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_0_0_0_0"
```

In the next step the database restores the data backup, one object for each volume is restored in parallel. After all data backup objects are restored, each volume continues with the restore of a set of log backup objects. Each volume restores multiple database objects with one or multiple calls. A server/volume will request multiple restore calls one after another, but you should expect multiple server/volumes to send restore requests in parallel on the same or other hosts.

```
backint -u BR1@BR1 -f restore -p aBackint.par -i /var/tmp/
backint_i_22xrk -o /var/tmp/backint_o_22xrk
```

Input file /var/tmp/backint_i_22xrk:

```
#SOFTWAREID "backint 1.50" "hdb indexserver 2.00.50"
#EBID "DF222xra" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_243987_246921"
#EBID "DF223lsw" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_246921_321035"
#EBID "DF224wm3" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_321035_338912"
#EBID "DF225wm3" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_338912_339977"
```

Result in output file /var/tmp/backint_o_22xrk:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"
#RESTORED "DF222xra" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_243987_2469 21"
#RESTORED "DF223lsw" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_246921_3210 35"
#RESTORED "DF224wm3" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_321035_3389 12"
#RESTORED "DF225wm3" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_338912_3399 77"
```

6.4 Inquiring Objects from Backint

HANA can inquire the state of the objects from the backint with "inquire" function.

```
backint -u BR1@BR1 -f inquire -p aBackint.par -i /var/tmp/
backint_i_gkn321 -o /var/tmp/backint_o_gkn322
```

Input file /var/tmp/backint_i_gkn321:

```
#SOFTWAREID "backint 1.50" "hdb nameserver 2.00.50"
#EBID "DF222xra" "/usr/sap/BR1/SYS/global/hdb/backint/
log_backup_6_0_243987_246921"
```

Result in output file /var/tmp/backint_o_gkn322:

```
#SOFTWAREID "backint 1.50" "ACME Corp backint 0815.2020.02"  
#BACKUP "DF222xra" "/usr/sap/BR1/SYS/global/hdb/backint/  
log_backup_6_0_243987_246921" "2020-02-06T08:02:44.100Z"
```

The backint tool must return a `#BACKUP` entry in the above format which contains the EBID, the object name and the creation timestamp